

Not All Differences Matter: Variability Exploration of Domain Models via Agentic AI

FirstName Surname[†]
Department Name
Institution/University Name
City State Country
email@email.com

FirstName Surname
Department Name
Institution/University Name
City State Country
email@email.com

FirstName Surname
Department Name
Institution/University Name
City State Country
email@email.com

ABSTRACT

Domain modeling is inherently interpretive: the same textual description may yield solutions that differ in structure, abstraction, and scope. Not all such differences constitute errors. Some reflect legitimate alternative representations and well-justified modeling decisions, while others stem from misconceptions or misapplied constructs. Existing evaluation approaches, however, tend to treat all deviations uniformly, thereby penalizing valid modeling competence and allowing real errors to pass undetected.

To address this gap, we present VEGO-AI (Variability Exploration for mOdelS via aGentic AI), an agentic framework that explicitly distinguishes between *substantial* (systematic and meaningful) and *occasional* (sporadic or erroneous) *variability*. VEGO-AI comprises four coordinated LLM-powered agents: (1) a *Language Advisor*; (2) a *Domain Advisor* that produces evolving reference guidelines capturing valid modeling alternatives; (3) a *Model Inspector* that assesses compliance while iteratively refining these guidelines; and (4) a *Variability Explorer* that identifies and classifies recurring deviation patterns.

We evaluated VEGO-AI in the context of a university-level modeling course. The results demonstrate that the framework successfully differentiates between substantial and occasional variability, offering a basis for model assessment and suggesting directions for rethinking evaluation practices in domain modeling.

CCS CONCEPTS

• Software and its engineering → Model-driven software engineering • Computing methodologies → Natural Language Processing.

KEYWORDS

Variability analysis, model-driven engineering, automated model assessment, generative AI, agentic AI, LLM

*Article Title Footnote needs to be captured as Title Note

[†]Author Footnote to be captured as Author Note

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

WOODSTOCK '18, June, 2018, El Paso, Texas USA

© 2018 Copyright held by the owner/author(s). 978-1-4503-0000-0/2005/06

ACM Reference format:

FirstName Surname, FirstName Surname and FirstName Surname. 2026. Insert Your Title Here: Insert Subtitle Here. In *Proceedings of ACM Woodstock conference (WOODSTOCK '18)*. ACM, New York, NY, USA, xx pages. <https://doi.org/10.1145/xxx>

1 Introduction

Domain models are the conceptual backbone of software systems, capturing requirements, structural relationships, and constraints that guide implementation. Yet, modeling is inherently interpretive: given the same domain description, different modelers may produce models that diverge in structure, abstraction, and scope. This divergence is not merely noise. It is, in fact, an interesting and practically consequential phenomenon in model-driven engineering. Consider the following three representative settings where this matters: (1) In product line engineering [23], practitioners deliberately introduce variability when designing generic models for families of related systems, yet not all variability is intentional or desirable; (2) In collaborative or research settings, when multiple stakeholders seek to converge on a shared domain model from a common specification [8], differences must be identified and reconciled, but only those that are substantively meaningful; and (3) In educational settings, students performing the same modeling task rarely produce models that match a reference exactly (nor should they be expected to), yet some deviations reflect misconceptions that must be identified and corrected.

Across all these settings, the critical challenge is the same: not all differences similarly matter. We draw a conceptual distinction between two fundamentally different types of deviation.

Substantial variability refers to differences that reflect contextually justified modeling choices – alternative yet valid representations of the same concept, optional decisions arising from underspecified aspects of the task, variations in scope, or alternative representations consistent with both the modeling language semantics and the domain logic. **Occasional variability**, on the other hand, denotes differences arising from errors, misconceptions, or unintended deviations, including syntactic violations, omission of explicitly required elements, misapplication of modeling constructs, or flawed conceptual understandings of the domain. Conflating the two types of variability is problematic: it

either penalizes valid modeling competence or allows real errors to pass undetected.

Current automated assessment tools, e.g., [5][6][29][4] are predominantly built on structural similarity metrics, comparing models against a single reference and treating all deviations similarly. While practical, these tools cannot reason about conceptual justifiability, nor can they aggregate deviation patterns to reveal systematic difficulties. Recent LLM-based approaches have made progress for model assessment [1][7][18], but monolithic LLM evaluation is prone to inconsistency and mixes language-level violations with domain-level misunderstandings. These limitations motivate our research, which addresses the following question:

To what extent can we automatically distinguish between substantial and occasional variability in domain models?

To this end, we present *VEGO-AI* (*Variability Exploration for mOdelS via aGentic AI*), a framework that operationalizes the distinction between substantial and occasional variability through a pipeline of four specialized LLM-powered agents: (1) The *Language Advisor* encodes the semantics of the target modeling language and serves as an oracle for language-related questions; (2) the *Domain Advisor* operationalizes domain requirements into evolving reference guidelines (rather than models), explicitly capturing alternative valid representations and serving as the authority on domain-related questions; (3) the *Model Inspector* assesses each model against the reference guidelines, producing a structured compliance vector; it may also identify previously unrecognized instances of substantial variability, triggering refinement of the guidelines; and (4) the *Variability Explorer* aggregates these vectors across models and guidelines to identify recurring deviation patterns and classify them as substantial or occasional variability. This separation of concerns ensures that language violations, domain misunderstandings, and valid modeling decisions are not confounded, enabling variability analysis that is both systematic and interpretable.

We evaluated VEGO-AI in the context of a university-level modeling course, focusing on student solutions to domain modeling tasks. Across two case studies and two modeling languages (UML use case and class diagrams), the framework generated reference guidelines comparable to those produced by the course staff, evaluated the student models with detailed feedback, and analyzed the variability across these models, contributing to the extension and refinement of the reference guidelines.

The core contributions of this work are threefold. First, we introduce a *conceptual distinction* between substantial and occasional variability. Second, we present *VEGO-AI*, an agentic framework that operationalizes this distinction through a modular four-agent pipeline. Third, we report *empirical findings* demonstrating how VEGO-AI can assist course staff in assessing and interpreting student modeling outcomes in educational settings. The rest of the paper is structured as follows. Section 2 provides background on variability and reviews related work on both LLM-assisted modeling and model evaluation. Section 3 describes VEGO-AI, and Section 4 presents and discusses the evaluation

results. Finally, Section 5 concludes the paper and outlines directions for future research.

2 Related Work

2.1 Variability

Variability is an inherent and well-recognized characteristic of systems. In product line engineering [21], variability is explicitly modeled to capture commonalities and differences among system variants and product families, supporting systematic reuse and configuration. Generally speaking, variability modeling [14][16][23] has been studied as a means to manage optionality, alternative design choices, and context-dependent behaviors in complex systems.

Different dimensions have been proposed to characterize variability. One fundamental distinction concerns binding time: *design-time variability* is bound during the engineering of a system or a product, whereas *runtime variability* allows a system to adapt and reconfigure itself during execution in response to changing contexts or requirements [16]. Another distinction addresses variability in space versus time: *variability in space* refers to concurrent variants of a system coexisting at a single point in time, whereas *variability in time* captures sequential variants arising from system evolution [3]. A further dimension concerns the origin of variability: *external (or essential) variability* stems from differences in stakeholder needs, deployment environments, or domain requirements, while *internal (or technical) variability* arises from design and implementation choices that are invisible to stakeholders [22]. Finally, variability can be distinguished based on intent: *accidental variability* refers to differences that are unintended by-products of the development process rather than consciously planned variation points, whereas *intentional variability* is explicitly modeled and governed [14], [16].

In business process modeling, different organizational contexts or regulatory environments require variants of the same underlying process. Here, too, several dimensions of process model variability have been identified [25]. With respect to *scope*, variability can occur in *control flow* (which activities are executed and in what order), in *resources* (which roles or organizational units perform activities), or in *objects* (which data and physical artifacts are produced or consumed). The *process type* dimension distinguishes between *conceptual* process models, intended for communication and reasoning about a domain, and *executable* models deployed on a Business Process Management System. The *customization type* dimension characterizes how variants are derived: by *restriction*, where a consolidated model representing the least common multiple of all variants is progressively constrained to yield a specific variant, or by *extension*, where a core model representing the greatest common denominator is augmented with context-specific elements.

Understanding variability as a multi-dimensional phenomenon makes clear that not every difference between models carries the same status. Some differences are intentional, contextually grounded, and semantically valid; they reflect legitimate modeling

or design choices within the space of acceptable solutions defined by the language, the domain requirements, and the task at hand (i.e., *substantial variability*). Others are unintended, erroneous, or non-compliant; they violate language rules, miss mandatory domain elements, or stem from misconceptions rather than deliberate decisions (namely, *occasional variability*). Operationalizing this distinction automatically is the core challenge this paper addresses.

2.2 Using LLMs for Modeling

The use of Large Language Models (LLMs) to support domain modeling has attracted growing research attention, with studies falling into two main streams: those that examine the produced *artifacts*, and those that examine the *modeling process*.

Artifact-centric studies assess LLM capabilities and limitations in producing correct models. Early work by Cámara et al. [10] established a baseline: unlike in code generation, LLMs struggle with modeling beyond very small diagrams (≈ 8 –10 classes), often misuse UML constructs such as association classes, multiple inheritance, and compositions, and are highly sensitive to domain and prompt phrasing. Bragilovski et al. [8] found that generative approaches outperform earlier non-generative ones in creating domain models, yet fall short of expert-produced models, with error analysis revealing introduction of redundant elements, roles, attributes, and implementation constructs. Calamo et al. [9] similarly found that LLM performance on UML class diagram generation is highly variable and dependent on model choice and prompting strategy; iterative prompting with feedback significantly improved correctness, though associations and cardinalities consistently prove harder to identify than classes. Chen et al. [12] confirm this pattern across multiple LLM families. Rajbhoj et al. [24] introduce LLM4Model, a configurable, domain-agnostic tool for automating model authoring, aimed at reducing cognitive load. Ferrari et al. [15] extended the scope to UML sequence diagrams, finding that structural correctness is achievable but capturing accurate behavioral semantics still requires substantial expert intervention.

Process-centric studies examine how humans and LLMs engage during the modeling process. Ali et al. [2] identified seven primary intentions shaping the interaction process (Create/Update Model, Create/Update List, Explain, Discuss, and Present) along with recurrent interaction patterns. Chaaben et al. [11] found that even with few-shot prompting for model completion tasks, human modelers remain essential for identifying and refining inaccuracies in LLM outputs, reinforcing the importance of deliberate collaboration design over pure automation. Silva et al. [27] proposed a human-in-the-loop approach in which a rule-based agent moderates a Q&A dialogue, selecting questions based on their potential to resolve the most uncertain aspects of the model, thereby enabling humans and LLMs to collaboratively refine it.

In process modeling, Klievtsova et al. [20] found that chatbot-based partial automation can be impactful even when substantial human refinement is still required, further underscoring that human involvement remains central to collaborative modeling regardless of the specific workflow.

Taken together, these studies converge on two consistent findings: LLMs are capable partners for the simpler structural elements of a model but remain unreliable for relational and behavioral elements, and human oversight is indispensable throughout. Neither finding, however, addresses how to interpret the variability that arises when multiple modelers – humans or LLMs – produce different solutions to the same modeling task.

2.3 Using LLMs for Model Evaluation

Automatically assessing the quality of models has also been studied extensively. Early approaches rely on heuristic rules and supervised machine learning (see for example [5][6][29][4][28]), providing fine-grained and explainable feedback but requiring substantial hand-crafted rules or reference models. Some more recent approaches leverage learned representations to complement rule-based techniques. Chen et al. [12] propose an embedding-based assessment approach that compares a candidate textual domain model against a reference solution using graph-edit distance and text embeddings. Singh et al. [28] focus on mistake detection in UML domain models, identifying 97 types of mistakes (e.g., plural class names, wrong multiplicities, misuse of player-role and abstraction-occurrence patterns) and automatically detect 83 of them using a rule-based metamodel and a comparison algorithm.

With the introduction and rapid growth of LLMs, model assessment approaches have increasingly shifted toward leveraging their reasoning and natural language understanding capabilities, either to complement or replace traditional rule-based and machine learning techniques. Ahmed et al. [1] introduce the MCeT tool, which evaluates sequence diagrams against informal requirements using an LLM-as-a-judge paradigm. To mitigate the fragility of holistic comparison, MCeT adopts a fine-grained, multi-perspective evaluation (diagram-atoms vs. requirement-atoms) and a cross-checking step to reduce hallucinated issues, substantially improving recall of human-reported issues while maintaining high precision. Bouali et al. [7] compare LLM-generated scores on UML class diagrams against teaching assistant grades, reporting strong alignment for GPT 4o-mini and Claude 3 Sonnet. Ibáñez et al. [18] similarly evaluate multimodal LLMs grading handwritten UML class diagrams against a rubric, aligned with the Open-Closed Principle. They report moderate agreement with experts for ChatGPT-4o, Gemini 2.5 Flash, and Claude 3.5 Sonnet, but also identify systematic reasoning errors (misinterpreting associations vs. inheritance, hallucinating methods or interfaces) and scoring inconsistencies.

Across both non-LLM and LLM-based evaluation approaches, three limitations recur. First, most approaches assess a model against a single correct reference model, treating every deviation as a defect and leaving no room for legitimate alternative solutions. Second, all tools are tailored to a specific modeling language, limiting generalizability. Third, and most directly relevant to this paper, none of the approaches distinguishes between deviations that reflect real modeling errors and those that reflect valid, context-driven modeling choices. Addressing this gap requires moving beyond generative AI to agentic AI – systems in which multiple specialized LLM-powered agents perceive, reason, plan, and act in

coordinated pipelines rather than responding to isolated prompts [19]. This direction has been advocated for software design and modeling specifically [13], and demonstrated in practice for requirements-to-deliverable pipelines that include UML diagrams [26].

3 The VEGO-AI Framework

VEGO-AI operationalizes the substantial/occasional variability distinction through a pipeline of four specialized LLM-powered agents, whose inputs, outputs, and communications are presented in Figure 1. A unifying design principle across all agents is the strict separation of language concerns from domain concerns: language-related questions (about construct semantics, syntactic ambiguity, or permitted construct usage) are always routed to the Language Advisor, while domain-related questions (about business rules, real-world facts, or the intended semantics of a domain element) are always routed to the Domain Advisor. This separation ensures that language violations, domain misunderstandings, and valid modeling choices are not mixed, enabling variability analysis that is both systematic and interpretable. We next elaborate on and demonstrate each agent’s functionality.

3.1 The Language Advisor (Agent 1)

The *Language Advisor* is responsible for all issues related to the target modeling language. It draws on an optionally provided formal language specification or reference manual, and on its own trained knowledge otherwise. The agent exhibits two skills.

Developing the Language Template. The agent produces a *Language Template* – a structured artifact that enumerates the language’s main elements, relations, and model fragments, one entry per distinct construct-related concern (T1, T2, ...).

The Language Template provides the shared vocabulary that grounds the subsequent work. Because the modeling language is well-defined and stable, the template is produced once and remains fixed throughout the framework’s execution. A partial example of a UML use case diagram template is provided below.

[{ "id": "T1", "short_name": "Actor", "fragment_description": "An actor represents a role that interacts with the use case within the system.", "involved_constructs": "Actor"},
{ "id": "T2", "short_name": "UseCase", "fragment_description": "A use case represents a functional requirement of the system, defining interactions between an actor and the system.", "involved_constructs": "UseCase"},
{ "id": "T3", "short_name": "Actor-Association-UseCase", "fragment_description": "An association between an actor and a use case demonstrates that the actor participates in the use case.", "involved_constructs": "Actor Association UseCase"},
{ "id": "T4", "short_name": "UseCase-Include-UseCase", "fragment_description": "An include relationship between two use cases indicates that behavior of the included use case is inserted into the base use case.", "involved_constructs": "UseCase Include UseCase"}...]

Answering Language-Specific Questions. The Language Advisor’s second skill is to answer language-specific questions escalated by other agents during the whole pipeline, for example, about naming equivalence, syntactic ambiguity, or permitted construct usage. Unlike the Language Template, which is produced once, this task is active throughout the entire pipeline, providing on-demand language grounding whenever another agent encounters an uncertainty it cannot resolve from the domain artifacts alone.

3.2 The Domain Advisor (Agent 2)

The *Domain Advisor* is responsible for all issues related to application domains. It draws on a provided *Domain Description* and on the *Language Template* produced by the Language Advisor. The agent possesses two skills.

Developing the Reference Guidelines. The Domain Advisor produces a structured set of *Reference Guidelines* grounded in the Language Template. Each guideline entry (G1, G2, ...) describes exactly one model fragment (a single element, a single relation, or a combination of such), maps it to one or more Language Template entries, and cites the exact segment (a phrase, a sentence, or a few related sentences) from the Domain Description that justifies it. Where the Domain Description permits multiple valid representations of the same concept, the guideline records these explicitly as alternatives rather than prescribing a single correct form. Any domain requirements that cannot be operationalized into guidelines are flagged explicitly. This design intentionally produces guidelines rather than a reference model: by abstracting away from a single correct solution, the Domain Advisor creates the space in which substantial variability can later be recognized rather than penalized. A partial example for a winery management domain is provided below.

[... { "id": "G2", "guideline_name": "Role of sales staff in placing orders", "description": "Sales staff are actors who interact with the system to place customer orders.", "related_template_id": "T1", "mapping_certainty": 0.9, "citation": "\"The sales staff are responsible for placing customer orders, with each order being assigned to a specific employee.\""}, ...
{ "id": "G12", "guideline_name": "Marketing department privileges", "description": "Represents a use case where only marketing department employees are actors who can update wine information.", "related_template_id": "T1", "mapping_certainty": 0.7, "citation": "\"Only employees in the marketing department are authorized to update wine information and types.\""}, ...
{ "id": "G25", "guideline_name": "Generating employee productivity report", "description": "A use case describing the generation of reports on unproductive employees based on their performance.", "related_template_id": "T2", "mapping_certainty": 0.8, "citation": "\"The 'Cheers' system will also generate the following reports: (1) Unproductive Employees Report: Generated by a sales employee, this report takes a time period as a parameter and displays the personal information of employees who placed fewer than 2 urgent orders OR fewer than 4 regular orders within that period;\""}, ...]

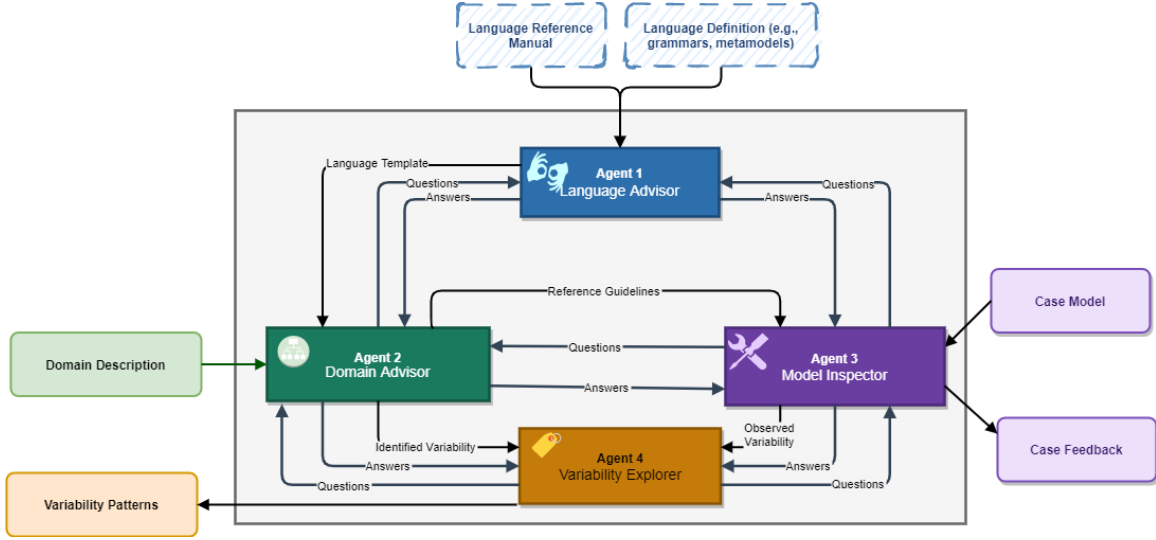


Figure 1. The VEGO-AI Architecture

Unlike the Language Template, the Reference Guidelines are not static. When the Model Inspector (agent 3) surfaces a modeling fragment that correctly represents a domain requirement but is not yet captured in the guidelines, the Domain Advisor is asked to evaluate whether the suggested fragment constitutes a valid representation, and if so, to update the affected guideline entry, extending its alternatives. This refinement loop means that the Reference Guidelines are evolved and progressively updated as models are inspected.

Answering Domain-Specific Questions. The Domain Advisor also responds to domain-related questions escalated by the Model Inspector during the evaluation pipeline, for example, about the intended semantics of a domain element, the scope of a business rule, or whether two domain concepts are equivalent. Like the Language Advisor's question-answering task, this task is active throughout the entire pipeline. Every answer is grounded in the Domain Description and the current guidelines; confidence is downgraded explicitly when neither source provides a definitive answer.

3.3 The Model Inspector (Agent 3)

The *Model Inspector* is responsible for evaluating individual *Case Models* against the Reference Guidelines and for contributing to their progressive refinement. It receives a Case model and the current Reference Guidelines, and performs three sequential tasks.

Mapping Existing Fragments. The Model Inspector maps each Reference Guideline to existing fragments in the Case Model, considering any alternative representations already recorded. Consequently, each guideline is labeled as either: *Satisfied*, *Partially-Satisfied*, or *Not-Satisfied*. Semantic naming flexibility is applied: a model element whose name differs from the guideline

but is semantically equivalent in the domain is considered compliant.

Discovering Potential Fragments. For guidelines not yet satisfied, the agent searches the Case Model for fragments that are not captured in the mapping but may nonetheless correctly represent the same domain requirement. Where the validity of a candidate cannot be determined from the available artifacts, the agent escalates to the Language Advisor or the Domain Advisor as appropriate. Any newly discovered valid alternatives are forwarded to the Domain Advisor to trigger the refinement cycle described in Section 3.2.

Auditing Uncovered Fragments. Finally, the agent identifies all model fragments in the Case Model not accounted for by any guideline and classifies each as one of three types: an *Alternative* (a valid modeling choice not required by the current guidelines), a *Domain Mistake* (contradicting the domain description or encoding an incorrect business rule), or a *Language Mistake* (misusing the modeling language syntactically). This stage is essential for detecting errors invisible to guideline-based checking alone: a model may satisfy all guidelines while still containing erroneous fragments reflecting misconceptions about the domain or the language. Alternatives discovered here are also forwarded to the Domain Advisor, either feeding the refinement cycle or flagged as case-specific.

Figure 2 illustrates an example output of the Model Inspector. The left side shows the Case Model (a use case diagram in this example), while the right side presents VEGO-AI's assessment. This includes the compliance vector, which categorizes each guideline as *Satisfied*, *Partially-Satisfied*, or *Not-Satisfied*, along with detailed evidence for individual fragments of the model (bottom right).

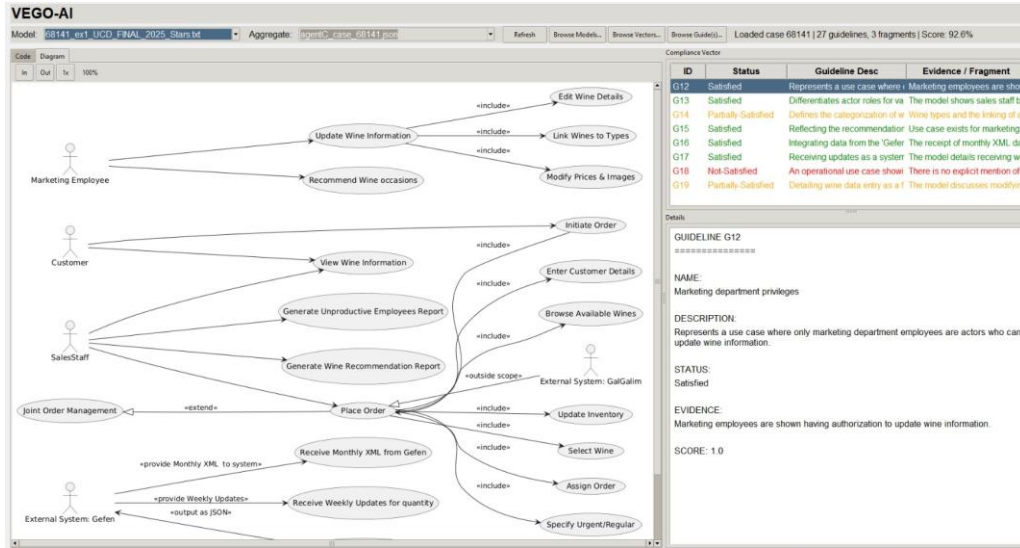


Figure 2. An example of an outcome of the Model Inspector agent

3.4 The Variability Explorer (Agent 4)

The *Variability Explorer* is responsible for reasoning about variability across the reference guidelines and the full set of evaluated models. It receives the outcomes of both Domain Advisor and Model Inspector and exhibits two skills.

Identifying Deviation Patterns. The Variability Explorer aggregates the per-case feedback and identifies recurring deviation patterns – fragments, omissions, or alternative representations that appear consistently across multiple models rather than in isolated cases. This population-level perspective helps distinguish between errors and valid choices: deviations that appear consistently and coherently across multiple models are reevaluated, potentially raising additional language or domain-related questions for the Language and Domain Advisors.

Classifying Variability. For each recurring pattern, the agent determines whether it constitutes substantial or occasional variability. A pattern is classified as *substantial variability* if it reflects a contextually justified modeling choice – an alternative representation consistent with both the language semantics and the domain logic. It is classified as *occasional variability* if contradicting evidence is found, indicating an error, misconception, or unintended omission, regardless of how frequently it recurs.

The **final output** is a set of variability patterns. As illustrated in Figure 3, every pattern is presented as a card comprising four elements: a classification badge indicating whether the pattern represents *substantial* or *occasional* variability, a confidence level reflecting the agent's certainty in the classification, a justification describing the observed modeling behavior and its interpretation, and an evidence field citing the specific guideline or domain statement from which the pattern was derived. Patterns classified

as substantial variability (P4 and P5 in the example) are additionally flagged for guidelines update, signaling that the reference guidelines may need to be revised to accommodate the observed modeling choice as a valid alternative.

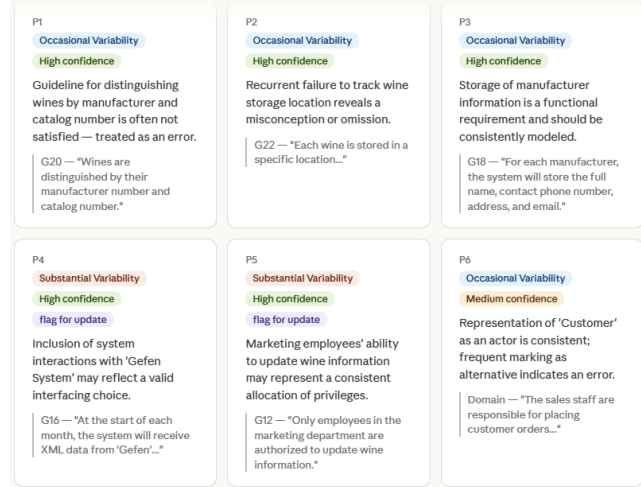


Figure 3. An example of the outcome of the Variability Explorer agent

4 Evaluating VEGO-AI

The framework was implemented as an asynchronous, multi-agent pipeline in Python, built around four specialized LLM-powered agents coordinated by a central orchestrator¹. All LLM calls are routed through a shared asynchronous client wrapping the OpenAI SDK, which handles authentication, token budgeting, and response

¹ The experimental material, including the framework and the evaluation pipeline implementation, is uploaded to EasyChair for the review process. If accepted, it will be uploaded to Zenodo.

parsing. Cross-agent communication is mediated through a structured Q&A protocol in which downstream agents raise globally scoped questions that are answered by the appropriate upstream expert agent, with the orchestrator enforcing a maximum round limit to prevent unbounded loops.

We further experimented with VEGO-AI in a university-level modeling course. We next elaborate on the evaluation goals, experimental design, procedures, results, and threats to validity.

4.1 Goals

The evaluation aims to address the following questions:

RQ1. To what extent are the generated language templates stable and aligned with expert-curated ground-truth?

RQ2. To what extent are the generated reference guidelines stable and aligned with expert understanding?

RQ3. To what extent are the scores derived from the generated case evaluation aligned with scores assigned by junior graders and experts?

RQ4. To what extent does VEGO-AI correctly classify variability in domain models?

4.2 Experimental Design

We evaluated VEGO-AI across two case studies and two modeling languages. The application domains are: *Cheers (ch)* – a winery management system encompassing inventory, orders, and customer management, and *ParkWise (pw)* – a nationwide parking lot management system. Both cases were drawn from the same second-year undergraduate modeling course in consecutive years and are comparable in complexity. Full domain descriptions are available with the experimental material. The modeling languages were *UML class diagrams (cd)*, the most widely used structural modeling notation in both research and practice, and *UML use case diagrams (ucd)*, which add a functional perspective to the evaluation. These choices of domains and languages yield four evaluation settings in total (two domains $\times$ two languages), as shown in Table 1. In all settings, the students’ models are provided in PlantUML format.

4.3 Procedures

To assess the quality and stability of the framework’s outputs, a dedicated three-phase evaluation pipeline was developed. In *Phase A*, the Language Advisor is executed three times from identical inputs to produce independent output variants; semantically similar outputs across runs are then clustered using semantic similarity, and each cluster is aligned to an expert-curated list of language constructs. The list for UCD, for example, includes use cases, actors, inheritance between actors, inheritance between use cases, `<<include>>` dependencies, `<<extends>>` dependencies, associations between use cases and actors, and system boundaries. Standard information-retrieval metrics (Precision, Recall, and F1) are computed to quantify coverage and correctness, while a cross-run agreement score (between 0 and 1) measures output stability. In *Phase B*, for each of the four settings, the Domain Advisor is executed three times using the corresponding language template with the highest F1 score from Phase A. The reference guidelines

produced in the three runs are clustered and assessed by an expert. Each cluster of guidelines receives a value: *Fully*, *Partially*, or *Wrongly* aligned with the domain description, resulting in a score between 0 and 1 for each setting.

Table 1. Evaluation Settings

Code	Domain	Language	Case Models	Description Length
ch-ucd	Cheers	UCD	46	741 words
ch-cd	Cheers	CD	47	
pw-ucd	ParkWise	UCD	44	986 words
pw-cd	ParkWise	CD	41	

In *Phase C*, the Model Inspector is executed on all case models using a set of Reference Guidelines produced in Phase B. The resulting compliance vectors and uncovered fragment audits are scored against a transparent, configurable scoring schema that assigns additive contributions to satisfied guidelines and penalizes errors, as shown below.

```
# Compliance statuses
Satisfied +1.0 / Guideline fully met by the case model
Partially-Satisfied +0.5 / Guideline partially met, some evidence present
Not-Satisfied 0.0 / Guideline not met, no evidence found
# Uncovered fragments
Alternative +0.5 / Valid alternative modeling choice, not in guidelines
Domain Mistake -1.0 / Fragment contradicts or misrepresents the domain
Language Mistake -0.5 / Fragment uses the modeling language incorrectly
```

The scores are compared to grades given by a junior grader in the course setting. A sample of the Model Inspector outcomes is further evaluated by two experts with extensive experience in teaching and assessing modeling artifacts, who judge the relevance of the generated feedback.

Finally, the variability classification (i.e., distinguishing substantial from occasional variability) produced by the Variability Explorer is qualitatively assessed by the authors.

The evaluation pipeline was executed for each one of the four settings (ucd_ch, cd_ch, ucd_pw, cd_pw).

4.4 Results

RQ1. Language Templates Stability and Alignment

Table 2 presents the agreement levels and the Precision (P), Recall (R), and F-measure (F) scores across three independent runs for the four evaluation settings. The Language Advisor demonstrated good stability, with cross-run agreement levels ranging from 0.78 (pw-ucd) to 0.875 (ch-ucd). When evaluated against the human-curated ground truth, the generated Language Templates yielded high performance, achieving perfect F-scores (1.0) in certain runs (e.g., Run 2 for ch-ucd and Run 1 for pw-ucd). The lowest observed F-score was 0.75 (Run 2 for pw-cd), illustrating a case in which the

agent reliably extracts core language templates, but misses some relations or dependencies.

RQ2. Reference Guidelines Stability and Alignment

The Reference Guidelines produced by the Domain Advisor for the three runs showed strong stability and good alignment with the expected domain logic. The average scores of the guideline clusters were consistently high across most settings: 0.85 for ParkWise UCD (pw-ucd) and ParkWise CD (pw-cd), and 0.88 for Cheers CD (ch-cd). The Cheers UCD (ch-ucd) setting yielded a lower average score of 0.70, potentially because the domain description text in this case tightly coupled functional requirements with structural constraints.

Table 2. Phase A Results

Setting	Agreement Level	Run 1 P/R/F	Run 2 P/R/F	Run 3 P/R/F
ch-ucd	0.88	1.0/0.88/0.93	1.0/1.0/1.0	1.0/0.75/0.86
ch-cd	0.83	0.88/0.78/0.82	1.0/0.78/0.88	0.9/1.0/0.95
pw-ucd	0.78	1.0/1.0/1.0	0.75/0.86/0.8	1.0/0.86/0.92
pw-cd	0.79	0.88/0.79/0.82	0.86/0.68/0.75	0.82/1.0/0.9

RQ3. Case Model Score and Feedback

Figure 4 illustrates the distribution of the scores generated by the Model Inspector agent (x-axis) and the grades assigned by the junior grader (y-axis). It is important to note that these manual evaluations do not represent an absolute ground truth, as they were provided by a grader with limited experience in modeling variations. Consequently, she relied heavily on a strict index where creativity was penalized. VEGO-AI, on the other hand, penalized redundancies and incorrect additions that the grader often overlooked, as they did not appear in the grading index. Nevertheless, we can observe a moderate positive correlation between the two scoring approaches.

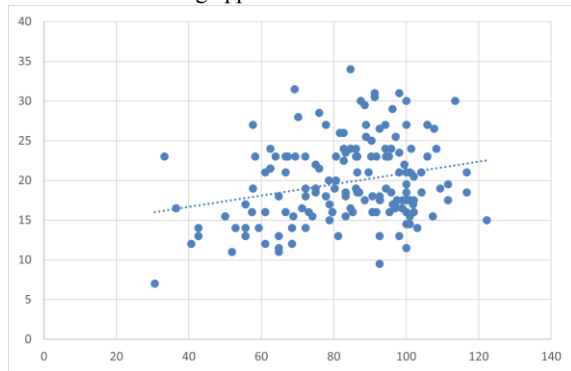


Figure 4. Model Scores Distribution

A sample of 16 Model Inspector outcomes (4 per setting) was reviewed by two experts, who evaluated both compliance vectors and uncovered audits. The score averages the rating across all

feedback items per case model, where each item is rated as fully, partially, or wrongly identified.

As shown in Table 3, the average scores on compliance vectors are consistently high across all four settings (0.80-0.96), indicating that the Model Inspector reliably assesses compliance. The average score on uncovered audits, on the other hand, is lower and more variable (0.55-0.88), reflecting the greater interpretive effort required to judge whether model additions are acceptable in specific contexts. The notably weaker scores in the use case diagram settings (ch-ucd and pw-ucd, both 0.55) suggest that uncovered fragment detection is more challenging in less structurally constrained languages and may require human involvement.

Table 3. Phase C Results

Setting	Compliance Vectors	Uncovered Audits
ch-ucd	0.80	0.55
ch-cd	0.96	0.81
pw-ucd	0.83	0.55
pw-cd	0.92	0.88

RQ4. Variability Patterns Identification and Classification

The Variability Explorer yielded 26 variability patterns across the four settings, as shown in Table 4. Eight of the 26 patterns were classified as substantial variability, all of which were flagged for guidelines update, and 18 as occasional variability. The patterns indicated as substantial variability were judged correct by the authors in all cases. These patterns flagged for guidelines update and were reviewed by the *Domain Advisor* in a subsequent pass, leading to targeted refinements of the reference guidelines.

Table 4. Explored Variability Patterns

Setting	# Substantial	# Occasional	Valid alternatives
ch-ucd	2	6	Gefen system interaction, marketing department as a distinct actor
ch-cd	2	2	External delivery scoping, Season enumeration for wine recommendations
pw-ucd	4	3	IRS actor, expanded controller role, SMS/UI delivery modes
pw-cd	0	7	All additions/ omissions reflect systematic errors in the domain

The patterns classified as occasional variability reveal three issues. The first is *guideline-expectation mismatch*: some guidelines presuppose a modeling decision that students consistently did not make, for instance, the extent to which operations are expected in a class diagram. The second is *cross-language contamination*: certain guidelines correspond to statements in the domain

description that are more naturally expressed in the other modeling language; students tended to omit the relevant elements, likely because the construct felt out of place in the language at hand. The third is *shared conceptual errors*: deviations recurring across many models reflect gaps in students' understanding of domain modeling, and as such serve as actionable signals for the course staff to revisit specific topics.

4.5 Threats to Validity

We discuss threats to validity following the standard categorization of construct, internal, external, and conclusion validity.

Construct validity concerns whether our measures accurately reflect the concepts they are intended to capture. For RQ1, the Language Template is evaluated against an expert-curated list of constructs using standard precision, recall, and F1 metrics. Although this list reflects human judgment, it was prepared independently of the agent's output and grounded in the formal language specification. For RQ2 and RQ3, expert evaluation follows a systematic protocol; some subjectivity is inherent here, as no formal ground truth for reference guidelines exists. For RQ4, variability classification was assessed qualitatively by the authors, again due to the absence of an established ground truth. Across all four RQs, the lack of objective ground truth is not a methodological gap but an intrinsic property of the tasks – expert judgment, applied systematically and independently, is hence appropriate.

Internal validity concerns whether the observed results are attributable to VEGO-AI rather than confounding factors. The pipeline is iterative: errors or omissions introduced by the Language Advisor (RQ1) propagate to the Domain Advisor (RQ2), which in turn affect the Model Inspector (RQ3) and ultimately the Variability Explorer (RQ4). Mitigation steps include generating reference guidelines three times and validating through expert clustering, and randomly sampling 16 models for RQ3 expert review. More broadly, these dependencies expose a key limitation of a fully automated pipeline: the findings suggest that human-in-the-loop oversight at key stages (guideline validation, compliance review, and variability classification) is necessary to ensure reliability at scale.

External validity concerns the generalizability of our findings. Our evaluation is limited to four settings, a single undergraduate modeling course at one institution and one LLM. The two cases are comparable in complexity, but both represent relatively bounded, well-scoped domains; results may differ for larger or more ambiguous domain descriptions. The framework was evaluated only with UML use case and class diagrams; whether the approach generalizes to other modeling languages, in particular behavioral ones such as sequence and state diagrams, remains an open question. Finally, all results are specific to the LLM used; different models may produce systematically different language templates, guideline structures, compliance labels, or variability classifications.

Conclusion validity concerns the reliability of the conclusions drawn from the data. For RQ1 and RQ2, conclusions rest on quantitative metrics computed over a small number of runs (three per setting). For RQ3, alignment with junior graders and experts is

encouraging, though robustness would require multiple graders and a larger annotated sample. For RQ4, classification was assessed qualitatively, as no quantitative benchmark exists.

5 Conclusion

This paper introduces a conceptual distinction between *substantial variability* – contextually justified modeling choices consistent with language semantics and domain logic, and *occasional variability* – errors, misconceptions, and unintended omissions. We presented **VEGO-AI**, an agentic framework that operationalizes this distinction through a modular pipeline: a **Language Advisor** grounding the pipeline in modeling language semantics, a **Domain Advisor** producing evolving reference guidelines that explicitly recognize valid alternatives, a **Model Inspector** evaluating individual models and feeding newly discovered alternatives back into the guidelines, and a **Variability Explorer** classifying recurring deviation patterns as substantial or occasional. Evaluated in a university-level modeling course across two domains and two modeling languages, VEGO-AI produced stable and accurate language templates and reference guidelines, with model scoring and variability classification aligning well with expert assessment. Beyond the educational setting, the framework can be applicable wherever model variability must be interpreted rather than merely detected, including product line engineering and collaborative modeling, though validation in those settings remains future work. The evaluation is bounded by its scope: two domains from a single institution, one LLM, and two modeling languages. Future work should extend the framework to behavioral modeling languages, such as sequence and state diagrams, where the space of valid alternatives is larger, and the boundary between substantial and occasional variability may be less clear-cut. In addition, incorporating human-in-the-loop oversight at key pipeline stages (guideline validation, compliance review, and variability classification) would strengthen reliability and is a natural next step toward deployment in real assessment settings.

REFERENCES

- [1] Ahmed, K., Song, J., Chen, B., Wei, O., & Zheng, B. (2025). MCeT: Behavioral Model Correctness Evaluation using Large Language Models. *Proc. ACM/IEEE 28th MODELS*, pp. 84–95.
- [2] Ali, S. J., Reinhartz-Berger, I., & Bork, D. (2024, October). How are LLMs used for conceptual modeling? An exploratory study on interaction behavior and user perception. In *International Conference on Conceptual Modeling* (pp. 257–275). Cham: Springer Nature Switzerland.
- [3] Ananieva, S., Greiner, S., Kehrer, T., Krüger, J., Kühn, T., Linsbauer, L., Grüner, S., Koziol, A., Lönn, H., Ramesh, S. and Reussner, R. (2022). A conceptual model for unifying variability in space and time: Rationale, validation, and illustrative applications. *Empirical Software Engineering*, 27(5), 101.
- [4] Auxepales, L., Py, D., & Lemeunier, T. (2008, July). A diagnosis method that matches class diagrams in a learning

- environment for object-oriented modeling. In 2008 Eighth IEEE International Conference on Advanced Learning Technologies (pp. 26-30). IEEE.
- [5] Bian, W., Alam, O., and Kienzle, J. (2019). Automated grading of class diagrams. In 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELSC), pages 700–709. IEEE.
- [6] Bian, W., Alam, O., and Kienzle, J. (2020). Is automated grading of models effective? assessing automated grading of class diagrams. In Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, pages 365–376.
- [10] Cámara, J., Troya, J., Burgueño, L., & Vallecillo, A. (2023). On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling*, 22(3), 781–793.
- [11] Chaaben, M. B., Burgueño, L., David, I., & Sahraoui, H. (2024). On the utility of domain modeling assistance with large language models. arXiv preprint arXiv:2410.12577.
- [12] Chen, K., Chen, B., Yang, Y., Mussbacher, G., & Varró, D. (2024). Embedding-based Automated Assessment of Domain Models. *Proc. MODELS Companion '24*, pp. 87–94.
- [13] Dam, H. K. (2025). Towards Multi-Agent AI for automated software design and modelling. *Proc. ASEW 2025*, pp. 311–314.
- [14] El-Sharkawy, S., Yamagishi-Eichler, N., & Schmid, K. (2019). Metrics for analyzing variability and its implementation in software product lines: A systematic literature review. *Information and Software Technology*, 106, 1–30.
- [15] Ferrari, A., Abualhaija, S., & Arora, C. (2024, June). Model generation with LLMs: From requirements to UML sequence diagrams. In 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW) (pp. 291-300). IEEE.
- [16] Galster, M., Weyns, D., Tofan, D., Michalik, B., & Avgeriou, P. (2013). Variability in software systems – a systematic literature review. *IEEE transactions on software engineering*, 40(3), 282-306.
- [17] Hamann, M., Götz, S., & Aßmann, U. (2024). Towards an Interoperable Model-driven Automated Assessment System for Computer Science Education. *Proc. MODELS Companion '24*, pp. 95–102.
- [18] Ibáñez, M. B., Barrón-Estrada, M. L., & Zatarain-Cabada, R. (2025). Can multimodal large language models grade like an expert? A study on UML class diagram assessment accuracy. *Computer Applications in Engineering Education*, 33(5).
- [19] Jin, H., Huang, L., Cai, H., Yan, J., Li, B., & Chen, H. (2024). From LLMs to LLM-based agents for software engineering: A survey. *arXiv:2408.02479*.
- [7] Bouali, N., Gerhold, M., Rehman, T. U., & Ahmed, F. (2025). Toward Automated UML Diagram Assessment: Comparing LLM-Generated Scores with Teaching Assistants. *Proc. CSEDU*, pp. 158–169.
- [8] Bragilovski, M., van Can, A. T., Dalpiaz, F., & Sturm, A. (2025). Leveraging machines to derive domain models from user stories. *Requirements Engineering*, 30(2), 241-262.
- [9] Calamo, M., Mecella, M., & Snoeck, M. (2025, June). Assessing the Suitability of Large Language Models in Generating UML Class Diagrams as Conceptual Models. In International Conference on Business Process Modeling, Development and Support (pp. 211-226). Cham: Springer Nature Switzerland.
- [20] Klievtsova, N., Benzin, J.-V., Kampik, T., Mangler, J., & Rinderle-Ma, S. (2023). Conversational process modelling: State of the art, applications, and implications in practice. In Business Process Management Forum (BPM 2023), pp. 319–336. Springer.
- [21] Metzger, A., & Pohl, K. (2014). Software product line engineering and variability management: achievements and challenges. *Future of software engineering proceedings*, 70–84.
- [22] Pohl, K., Böckle, G., & Van Der Linden, F. (2005). *Software product line engineering: foundations, principles, and techniques* (Vol. 1). Heidelberg: Springer.
- [23] Pol'la, M., Buccella, A. & Cechich, A. Analysis of variability models: a systematic literature review. *Softw Syst Model* 20, 1043–1077 (2021).
- [24] Rajbhoj, A., Somase, A., Sant, T., Vale, S., & Kulkarni, V. (2025, June). LLM4Model: Automated Requirements Specification Model Authoring. In International Conference on Advanced Information Systems Engineering (pp. 128-136). Cham: Springer Nature Switzerland.
- [25] Rosa, M. L., Aalst, W. M. V. D., Dumas, M., & Milani, F. P. (2017). Business process variability modeling: A survey. *ACM Computing Surveys (CSUR)*, 50(1), 1-45.
- [26] Sami, M. A., Waseem, M., Rasheed, Z., Saari, M., Systä, K., & Abrahamsson, P. (2024). Experimenting with multi-agent software development: Towards a unified platform. *arXiv:2406.05381*.
- [27] Silva, J., Ma, Q., Cabot, J., Kelsen, P., & Proper, H. A. (2025, October). Towards Human-in-the-Loop LLM-Enabled Domain Modeling. In International Conference on Conceptual Modeling (pp. 127-145). Cham: Springer Nature Switzerland.
- [28] Singh, P., Boubekeur, Y., & Mussbacher, G. (2022). Detecting mistakes in a domain model. *Proc. MODELS '22 Companion*, pp. 257–266.
- [29] Tselonis, C., Sargeant, J., & Wood, M. M. (2005). Diagram matching for human-computer collaborative assessment. *Proceedings of the 9th International Computer Assisted Assessment Conference* (May 2005).